UNIVERSIDAD NACIONAL DE EDUCACIÓN A DISTANCIA

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Proyecto de Fin de Grado en Ingeniería Informática

Entorno de Desarrollo Integrado de Python

Daniel de la Fuente Matamala

Dirigido por: José Félix Estívariz López

Curso: 23/24 convocatoria extraordinaria de diciembre

Entorno de Desarrollo Integrado de Python

Proyecto de Fin de Grado en Ingeniería Informática de modalidad Genérica

Realizado por: Daniel de la Fuente Matamala

Dirigido por: José Félix Estívariz López

Fecha de lectura y defensa: <FECHA DEFENSA>

Resumen

Esta memoria documenta la creación de un entorno de desarrollo integrado (IDE) específico para Python, diseñado para facilitar las operaciones fundamentales de edición, ejecución y depuración de código.

El proyecto se estructura en capítulos que abarcan desde el análisis inicial hasta la valoración del producto final.

Tabla de contenidos

Resumen……3

Tabla de Contenidos 4

Introducción…….……..……..……5

Estado del Arte……. ……. ……6

Objetivos del PFG……..9

Metodología y Planificación……13

Inicio del Desarrollo……..……..……..……...18

Análisis.. ……..……..……..……19

Diseño……..……..……..……23

Codificación……..……..……..……31

Conclusiones……..……..……..……40

Mejoras y Trabajos futuros……..……..44

Referencias……..……..……..……45

Introducción

Un entorno de desarrollo integrado (IDE) específico de un lenguaje es una herramienta que optimiza y facilita el desarrollo de software en un lenguaje de programación en particular, proporciona a los desarrolladores un conjunto de herramientas para el desarrollo en una sola interfaz. A diferencia de un IDE de propósito general está limitado a un solo lenguaje de programación o tecnología[i].
Un IDE específico de un lenguaje de programación, proporciona las siguientes características[ii]:

1. **Lenguaje:** Soporte para un lenguaje de programación en específico, en este caso, **Python**.

2. **Interfaz de usuario intuitiva:** Que facilite la navegación a través de las herramientas y características del IDE y por el código (resaltado de sintaxis).

3. **Editor de código:** Que permita escribir, editar y formatear el código de manera eficiente.

4. **Gestión de proyectos:** Capacidad para organizar ficheros y directorios relacionados con un mismo desarrollo.

5. **Funciones de edición habituales:** Cortar, copiar, pegar, seleccionar todo, deshacer y rehacer.

6. **Ejecución:** Ejecutar el código y redirigir la salida o errores a un terminal de resultados.

7. **Depuración:** Ejecutar el código paso a paso.

8. **Manejo de ventanas múltiples:** De edición y resultados.

9. **Análisis estático del código:** Identificar posibles problemas o errores en el código y señalarlos.

10. **Configuración del entorno:** Directorios para la gestión de proyectos y aspecto (estilo) del interfaz.

11. **Integración de herramientas externas:** Soporte nativo para bibliotecas externas, frameworks, control de versiones, etc.

12. **Gestión de plantillas:** Para automatizar, estandarizar el código y, por tanto, aumentar la productividad.

13. **Ayuda:** Manual de usuario, asistencia y resolución de problemas.

Estado del Arte

Prospección de soluciones existentes (comerciales y abierta) de IDEs específicos de lenguaje, IDEs específicos de Python:

I. **Pycharm (JetBtrains)[iii]: IDE comercial** específicamente diseñado para el desarrollo en Python. Es uno de los entornos más populares y robustos para Python, se diseñó con el objetivo de proporcionar un entorno altamente productivo, incorporando continuamente nuevas características que responden a las necesidades cambiantes de los desarrolladores de Python como soporte para ciencia de datos, desarrollo web y automatización.

   1) **Ventajas:**

      a) IDE generalista con enfoque fuerte en desarrollo de aplicaciones profesionales.

      b) Ofrece un entorno listo para usar, con configuraciones predeterminadas optimizadas para Python.

      c) Mejora de productividad al reducir errores comunes, optimizar la escritura de código e integrar herramientas externas lo que reduce el tiempo al cambiar entre distintas herramientas.

      d) Facilita la detección y corrección de errores complejos gracias al soporte de pruebas.

      e) Entorno completo para desarrollar y depurar aplicaciones web, muy útil para desarrolladores que trabajan en aplicaciones full-stack.

   2) **Limitaciones:**

      a) Alto consumo de recursos del sistema, puede ser un inconveniente al trabajar en sistemas menos potentes o con proyectos muy grandes.

      b) Curva de aprendizaje pronunciada para principiantes. A pesar de presentar una interfaz intuitiva tiene una gran cantidad de características y configuraciones disponibles lo que puede abrumar a los desarrolladores novatos.

c) Licenciamiento. PyCharm ofrece una edición gratuita pero las características más avanzadas están disponibles solo en la edición de pago. Un factor limitante para desarrolladores independientes o pequeñas empresas que no pueden permitirse el coste de la licencia comercial.

II. **Spyder[iv]: IDE de código abierto**, en un principio fue diseñado por la comunidad para facilitar las tareas de científicos, ingenieros y analistas de datos que utilizan Python, por lo que cuenta con características específicas orientadas a la ciencia de datos y análisis numérico.

1) **Ventajas:**

a) IDE que está específicamente orientado a la ciencia de datos y la programación científica, queda reflejado en características como el explorador de variables y la consola interactiva.

b) Diseño ligero.

c) Escritura del código libre de errores, eficiente que permite a los usuarios enfocarse en la lógica del análisis en lugar de en errores sintácticos.

d) Iteración rápida sobre el código, análisis y verificación de resultados de manera interactiva.

e) Permite generar visualizaciones directamente desde el entorno gracias a la integración con las herramientas científicas, lo que mejora el flujo de trabajo.

2) **Limitaciones:**

a) Spyder puede experimentar problemas de rendimientos cuando se trabaja con proyectos extremadamente grandes o con datos masivos. En parte debido a su diseño ligero.

b) Carece de algunas características avanzadas para el desarrollo profesional, como herramientas para desarrollo web.

c) Como herramienta de código abierto, depende en gran medida del apoyo de la comunidad para el desarrollo y mantenimiento, lo cual limita el soporte y actualizaciones.

III. Características comunes que implementan ambas soluciones:

1) **Editor de código:** Con autocompletado, análisis estático y refactorización de código

2) **Consola:** Para ejecuciones del código, scripts y pruebas y un terminal para ejecutar comandos del sistema.

3) **Depurador paso a paso:** Ejecuta el código paso a paso para identificar y corregir errores.

4) **Integración de herramientas externas:** Ofrece soporte nativo para una variedad de frameworks web, herramientas de ciencia de datos y sistemas de control de versiones.

5) **Gestión de proyectos:** Explorador de carpetas del sistema de archivos que contiene todos los ficheros y scripts relevantes para la tarea. El explorador de proyectos muestra la estructura de directorios y archivos del proyecto.

Objetivos del PFG

El objetivo de este PFG es construir un IDE específico de Python que facilite las operaciones fundamentales de la edición de software.

Características funcionales:

1. **Interfaz de usuario:** Una interfaz intuitiva que proporcione un entorno “listo para usar” con las herramientas para editar software “a la vista” o fácilmente navegables.

2. **Editor de código:** Con análisis estático del código, resaltado de sintaxis y de errores.

3. **Manejo de ventanas múltiples (edición y resultados):** Posibilidad de cambiar entre pestañas de edición y ejecución de programas distintos.

4. **Funciones de Edición habituales:** Cortar, copiar, pegar, seleccionar todo, deshacer y rehacer. Así como, crear nuevo programa, abrir uno existente, cerrar pestañas de edición y ejecución, guardar, guardar como… etc.

5. **Gestión de proyectos:** Un widget explorador de las carpetas del sistema de archivos que ofrezca la posibilidad de poder abrir los programas, renombrarlos, eliminarlos y copiar su ruta. Además ofrece la opción de volver a la carpeta superior o al directorio raíz.

6. **Ejecución del código:** Ejecutar el código y redirigir salida o errores a una ventana de resultados con estilo de terminal.

7. **Depurador paso a paso:** Ejecutar el código paso a paso, señalando la línea que se está ejecutando en este momento, redirigiendo los resultados o errores a la ventana de resultados y con las funciones de depuración habituales, resumir ejecución, siguiente línea, step y stop.

Características cualitativas:

I. **Usabilidad[v]:** Facilidad con la que los usuarios interactúan con el programa y sus funcionalidades. Para este proyecto se ha analizado la documentación de la ISO 9241 sobre el diseño centrado en el usuario y cuya aplicación van a permitir que las tareas que realice el usuario con la aplicación se realicen de manera efectiva, eficiente y satisfactoria.

   La usabilidad es alta, con una interfaz intuitiva con acciones claras y una disposición de widgets que separa eficazmente la zona de edición, visualización de resultados y gestión de proyecto.

   Los componentes a los que se les han aplicado principios de usabilidad son:

   1) **Editor de código:** Es el núcleo de la interfaz, es la función que más se va a utilizar.
      Para facilitar la comprensión de código el resaltado de sintaxis proporciona una separación visual de las distintas palabras y/o símbolos del lenguaje. Esto mejora la experiencia de usuario permitiéndole leer y escribir código de manera más clara.
      El resaltado de errores ayuda a los usuarios a visualizar errores de sintaxis directamente en el editor en tiempo real, evitando que se acumulen los errores antes de probar a ejecutar el código, lo que se traduce en una mayor productividad.

   2) **Explorador:** Proporciona una navegación intuitiva por el sistema de archivos, restringir la visualización a solo programas escritos en Python facilita la selección adecuada y simplifica el uso y las acciones comunes (abrir, renombrar, eliminar, copiar ruta) son bien entendidas por los usuarios, lo que mejora la familiaridad.

   3) **Ventanas múltiples:** Permite al usuario abrir múltiples archivos de código simultáneamente sin cambiar de ventana, facilita la navegación del sistema, identificando qué archivos están abiertos, cuáles tienen cambios sin guardar, mediante los títulos de las pestañas, mejorando la experiencia de usuario y la eficiencia.

   4) **Botones para generar resultados:** Botones para ejecutar o depurar el código y redirigir la salida a un terminal, fáciles de comprender y de usar, generan mensajes de error con comportamientos no permitidos, como pulsar el botón de ejecutar sin tener un programa abierto antes.

5) **Ventana de resultados:** Ventana con estilo de terminal. En modo ejecución imprime la salida de los programas y/o sus errores y en modo depuración se añaden unos botones para las operaciones de depuración habituales (resumir ejecución, siguiente línea, step, stop). Esta ventana de ejecución se proporciona en un espacio inferior al editor de código, creando un espacio para ver los resultados de forma separada.

6) **Barra de menús:** Con las operaciones de edición habituales, proporciona las funcionalidades esperadas como en otros programas, lo que reduce la curva de aprendizaje y aumenta la eficiencia.

II. **Mantenibilidad y flexibilidad[vi]:** Se han aplicado principios para que la organización de los componentes del programa en unidades separadas fáciles de mantener o actualizar. Para este proyecto se ha analizado la documentación de la ISO 25010 relativa al modelo de calidad del producto software.

En lo que a flexibilidad se refiere se ha buscado que el proyecto pueda ser adaptable, escalable y reemplazable, mientras que en el apartado de mantenibilidad se ha puesto el foco principalmente en la modularidad, que sea reusable y analizable, así como que tenga una alta capacidad para ser modificado y probado.

Los componentes a los que se les han aplicado principios de mantenibilidad y flexibilidad son:

1) **Editor de código:** Widget independiente, interactúa con la clase principal del programa para el análisis estático y el resaltado de errores, y con la ventana de resultados a través de la clase principal para redirigir la salida y/o errores.

2) **Explorador:** Componente autónomo, interactúa con el sistema para abrir un archivo en el editor, desacoplado del resto de widgets.

3) **Ventanas múltiples:** Las pestañas son un módulo independiente que sirve como contenedor para los editores de código, permite escalar el número de editores abiertos simultáneamente. Cualquier gestión de pestañas puede realizarse sin afectar directamente al editor de código. Interactúan con la clase principal para renombrar los títulos de las pestañas, si tienen cambios sin guardar, si renombramos un programa, también para mostrar un mensaje al tratar de cerrar una pestaña que contiene cambios sin guardar.

4) **Botones para generar resultados:** La funcionalidad está asociada a la activación de estos botones, es posible modificar o expandir su lógica sin afectar al resto de módulos.

5) **Ventana de resultados:** Interactúa con el programa principal para imprimir los resultados o errores de las ejecuciones o del propio programa, puede modificarse o expandirse sin afectar al resto de módulos.

6) **Barra de menús:** Es otro módulo que puede modificarse o ampliarse sin afectar a otros, interactúa con el editor de código con las funciones estándar de edición (cortar, copiar, pegar, etc.) y con la clase principal para las funciones de archivo (nuevo, abrir, guardar, etc.). Nuevas funcionalidades podrían añadirse sin cambiar significativamente la interfaz.

En general, la arquitectura del programa, al ser modular y bien desacoplada, permite un mantenimiento y adaptabilidad ante cambios de forma sencilla y segura.

Metodología y planificación

Metodología:

Modelo: el modelo elegido de ciclo de vida para el desarrollo de este proyecto es el modelo **de prototipos**[vii].

Se ha elegido este modelo debido a que sirve como mecanismo para identificar fácilmente los requisitos del software debido a las etapas que tiene su construcción. Cabe destacar que la etapa de la recolección de requisitos del enunciado propuesto, así como la realización de un primer diseño rápido centrado en los aspectos del software más básicos permite obtener resultados rápido, que con la evaluación de dicho prototipo y sus posteriores refinamientos permiten obtener el producto final.

Herramientas y tecnologías:

- **Qt Designer**[viii] (Prototipo 1): Se trata de una herramienta de Qt que proporciona una interfaz de usuario de tipo WYSIWYG, “what-you-see-is-what-you-get”, para crear GUIs para aplicaciones PyQT de forma eficiente.

- **Visual Studio, PyQt**[ix] (Prototipos 2-15): Se trata de una extensión para visual studio code, son las librerías que permiten a los desarrolladores escribir aplicaciones gráficas en Python utilizando las capacidades de Qt. .

1. **Prototipo:** Interfaz Gráfica de Usuario.

2. **Prototipo:** Editor de Código.

3. **Prototipo:** Conectar acciones “Archivo” y “Editar” de la barra de menús con el editor y ajustar el comportamiento de la interfaz.

4. **Prototipo:** Ejecución de programas y visualización de errores o resultados por la ventana de resultados.

5. **Prototipo:** Análisis estático y resaltado de errores de sintaxis.

6. **Prototipo:** Mejora del editor: Números de línea, highlight de línea actual y resaltado de sintaxis.

7. **Prototipo:** Explorador con operaciones.

8. **Prototipo:** Menú contextual con las acciones para los diferentes prototipos ya implementados.

9. **Prototipo:** Ventana de configuración/personalización.

10. **Prototipo:** Depuración de programas (ejecución paso a paso).

Estos prototipos se han definido de cara a la realización del diagrama de planificación, las descripciones de diseño y código de las partes añadidas en cada prototipo se definen en el capítulo de **Desarrollo.**

Planificación:

Para mostrar la secuenciación seguida para la realización de este proyecto se ha utilizado un **diagrama de Gantt**[x] que permite mostrar gráficamente el trabajo que se va a realizar así como las expectativas del tiempo requerido para cada uno de los prototipos explicados previamente.

–Diagrama de Gantt en la siguiente página–

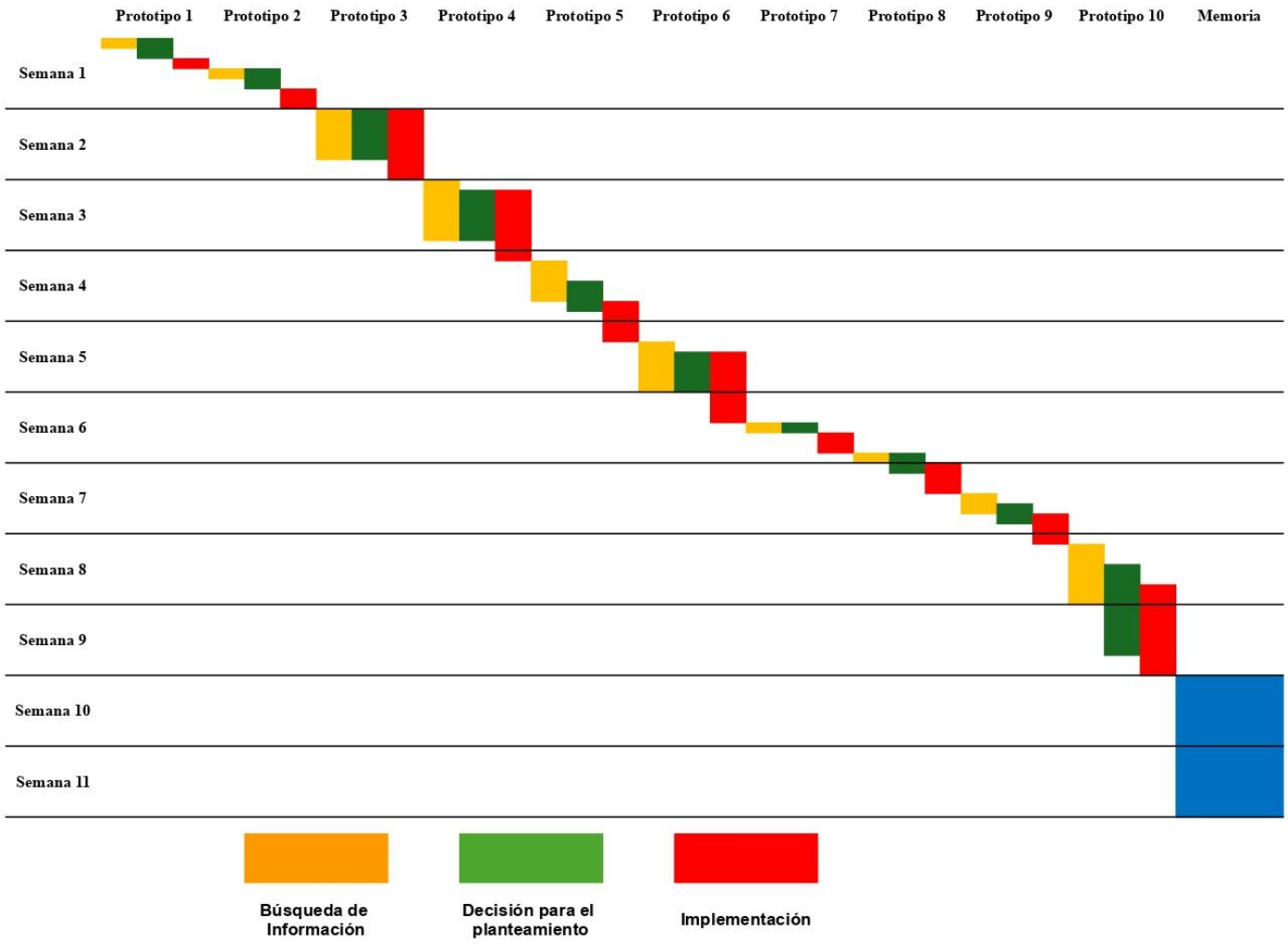


Ilustración 1: Diagrama de Gantt

Inicio del Desarrollo

De acuerdo con el modelo de ciclo de vida basado en prototipos, el desarrollo del IDE se abordará iterativamente, construyendo y refinando componentes clave recogidos en casos de uso y requisitos funcionales. Esto permite una evolución gradual, adaptada a las necesidades del usuario final:

- **Caso de Uso:** Crear o abrir un archivo Python y editarlo.
- **Caso de Uso:** Ejecutar un programa Python y visualizar los resultados o errores.
- **Caso de Uso:** Depurar el código Python.

Análisis

1. **Interfaz Gráfica de Usuario (GUI)**

   - **Historia de Uso:** Interactuar fácilmente con las herramientas del IDE y acceder a las funcionalidades eficientemente y sin complicaciones.

   - **Requisitos Funcionales:**
     - Barra de menús con opciones estándar (Archivo, Editar, Configuración).
     - Disposición de tres áreas principales:
       - Editor de código en el centro.
       - Explorador de proyectos en un panel lateral.
       - Ventana de resultados en la parte inferior.
     - Soporte para múltiples ventanas de edición (pestañas).
     - Botones claramente etiquetados para ejecutar y depurar programas.

2. **Editor de Código**

   - **Historia de Uso:** Un editor de código que facilite escribir y editar código Python con soportes que mejoren la productividad y reduzcan la aparición de errores tales como el resaltado de sintaxis y de errores.

   - **Requisitos Funcionales:**
     - Resaltado de sintaxis para palabras clave, variables, operadores, etc.
     - Análisis estático en tiempo real con resaltado de errores.
     - Operaciones estándar de edición: cortar, copiar, pegar, deshacer, rehacer, seleccionar todo.
     - Marcado de línea actual y de errores y número de línea en el margen.
     - Soporte para abrir, guardar, guardar como, y cerrar archivos.
     - Identificación visual de pestañas con cambios no guardados.

3. **Explorador de Proyectos**

- **Historia de Uso:** Posibilidad de navegar y gestionar archivos y carpetas de proyecto desde el IDE con operaciones como abrir, renombrar, copiar rutas o eliminar archivos directamente.

- **Requisitos Funcionales:**
    - Visualización de directorios y archivos de Python (*.py).
    - Operaciones de archivo:
        - Abrir archivos en el editor.
        - Renombrar o eliminar archivos.
        - Copiar la ruta completa del archivo al portapapeles.
    - Navegación entre carpetas, con accesos directos a un directorio raíz o al nivel superior.

4. **Consola y Ventana de Resultados**

- **Historia de Uso:** Facilitar la ejecución de código desde el IDE y visualización de los resultados o errores en una consola integrada, para no tener que alternar entre diferentes aplicaciones.

- **Requisitos Funcionales:**
    - Ejecución de código Python con salida redirigida a la ventana de resultados.
    - Visualización de errores de ejecución o mensajes del programa en tiempo real.
    - Botón para ejecutar programas.
    - Ventana de resultados estilo terminal ubicada debajo del editor de código.

5. Depuración Paso a Paso

- **Historia de Uso:** Identificar errores y evaluar el flujo del código en tiempo real mediante ejecución de un programa paso a paso.

- **Requisitos Funcionales:**
  - Botones para operaciones básicas de depuración:
    - Resumir ejecución.
    - Ejecutar siguiente línea.
    - Step (entrar en funciones).
    - Stop (detener depuración).
  - Indicación de la línea en ejecución.
  - Redirección de la salida y errores a la ventana de resultados.

Este enfoque modular y centrado en el usuario asegura que cada componente sea funcional y se pueda evaluar antes de avanzar a la siguiente fase del desarrollo.

Caracterización de Casos de Uso

Caso de Uso 1: Editar un Archivo Python, nuevo o existente

- **Actor:** Desarrollador.
- **Flujo Principal:**
  1. El usuario selecciona a través del explorador un archivo existente o en la barra de menús, ya sea mediante clics del ratón o atajos de teclado, “Archivo > Nuevo” o “Archivo > Abrir > Abrir archivo seleccionado”.
  2. Se abre una pestaña en blanco o se carga el contenido del archivo seleccionado en el editor de código.
  3. El usuario escribe código en el editor.
  4. El IDE marca y muestra, si los hubiera, los errores de sintaxis.
  5. El usuario guarda el archivo seleccionando “Archivo > Guardar o Guardar como”
  6. El sistema guarda el archivo en el directorio especificado.

Caso de Uso 2: Ejecutar un Programa

- **Actor:** Desarrollador.
- **Flujo Principal:**
  1. El usuario presiona el botón Ejecutar.
  2. El programa se ejecuta, y los resultados o errores se muestran en la ventana de resultados.
  3. Excepción: Si no hay ningún archivo abierto, el sistema muestra un mensaje de error.

Caso de Uso 3: Depurar Código

- **Actor:** Desarrollador.
- **Flujo Principal:**
  1. El usuario presiona el botón Depurar.
  2. El IDE muestra la primera línea de código a ejecutar.
  3. El usuario utiliza los botones Siguiente línea o Step para avanzar.
  4. Los resultados o errores se muestran en tiempo real en la ventana de resultados.
  5. El usuario detiene la depuración seleccionando Stop.

Diseño

Arquitectura General

La arquitectura **modular basada en capas**[xi] no solo es coherente con el modelo de ciclo de vida de prototipos, sino que es un enfoque ideal para maximizar sus beneficios.

Cada módulo se desarrolla y valida de manera independiente, minimizando el riesgo de errores que afecten al sistema completo y facilita realizar ajustes en una capa o módulo sin necesidad de modificar todo el sistema, garantizando un desarrollo eficaz y controlado.

1. **Capa de Presentación (GUI):** Contiene los elementos visuales de la aplicación, inicializada por la clase GUI. Los widgets específicos con sus operaciones, se encapsulan en clases propias:

    - GUI
    - Programa → AreaCodigo
    - Consola
    - Explorador
    - ConfigWindow

2. **Capa de Control:** Main inicia la aplicación y el controlador principal (Clase Controlador) actúa como mediador entre la interacción del usuario y la lógica de la aplicación. Gestiona los eventos de usuario, coordina la interacción entre los módulos, y actualiza la interfaz gráfica según sea necesario.

3. **Capa de Lógica de Aplicación:** Implementa el resaltado de sintaxis (Clase Highlighter) y el análisis estático del código (Clase NameChecker). Los datos de los programas que interactúan con la lógica de la aplicación se encapsulan en la clase Programa.

Descripción Detallada de los Componentes

1. **Clase Main**

   - **Función principal:** Punto de entrada del programa.
   - **Responsabilidad:** Inicializa la interfaz gráfica llamando a Controlador, que controla el flujo de la aplicación.
   - **Interacciones:** Llama al controlador para inicializar la aplicación y cargar la ventana principal con sus widgets.

2. **Clase GUI (Generado con Qt Designer)**

   - **Función principal:** Define la ventana principal de la aplicación y su estructura visual.
   - **Responsabilidades:** Proveer el diseño base de la interfaz.
   - **Interacciones:** Es controlada por Controlador para responder a eventos.

3. **Clase Controlador**

   - **Función principal:** Coordina los eventos, la lógica de la aplicación y las interacciones entre los módulos y la interfaz de usuario.
   - **Responsabilidades:**
     - Gestionar la lógica de eventos (e.g., clics de botones, cambios en los widgets).
     - Invocar métodos disparados por los eventos y de los módulos funcionales (AreaCodigo, Higlighter, Namechecker, etc.) según las acciones del usuario.
     - Actualizar la interfaz gráfica a través de interacciones con GUI.
   - **Interacciones:**
     - Recibe eventos de la GUI y los redirige a los módulos o métodos correspondientes.
     - Coordina la interacción con el resto de módulos.
     - Utiliza instancias de Programa para almacenar y recuperar datos asociados a los programas editados.

4. **Clase Programa**

- **Función principal:** Modelo de datos que representa un programa en edición.
- **Responsabilidades:**
  - Almacenar atributos como la ruta del programa, su pestaña de la GUI y AreaCodigo asociadas, su contenido y estado (e.g., si tiene cambios sin guardar).
  - Proveer getters y setters para acceder y modificar estos datos.
- **Interacciones**: Utilizada por Controlador para gestionar los programas abiertos.

5. **Clase AreaCodigo**

- **Función principal:** Implementa el widget del editor de texto y extiende su lógica y contenidos.
- **Responsabilidades:**
  - Extender funcionalidades como un widget de números de línea, resaltado de línea actual, resaltado de sintaxis (Highlighter) y para resaltar errores de nombres no declarados (NameChecker).
  - Emitir eventos para acciones realizadas dentro del editor (p.e., cambios en el texto).
- **Interacciones:**
  - Es controlada por Controlador para realizar acciones como cargar el código de un fichero existente, guardarlo, resaltado de sintaxis y errores, ejecutar el código o depurarlo.
  - Utiliza Higlighter para el resaltado de sintaxis.
  - Utiliza NameChecker para comprobar nombres no declarados.
  - Maneja eventos relacionados con los atajos de teclado y controlar el widget NumeroDeLinea.

6. **Clase Consola**

- **Función principal:** Modelo de datos que representa la salida de un programa en ejecución o depuración.
- **Responsabilidades:**
  - Almacenar atributos como la pestaña de la GUI y widget de texto.
  - Mostrar mensajes de salida y errores.
  - Integrarse con Controlador para actualizar la consola según los eventos de ejecución.
- **Interacciones:** Recibe mensajes del Controlador para imprimir resultados o errores en el widget de texto.

7. **Clase Explorador**

- **Función principal:** Gestionar la visualización y navegación por el sistema de archivos.
- **Responsabilidades:**
  - Navegar por el sistema de archivos y mostrar directorios y archivos relevantes.
  - Emitir eventos para abrir, renombrar, copiar ruta o eliminar archivos.
- **Interacciones:** Se comunica con Controlador para abrir archivos en el editor o realizar operaciones en el sistema de archivos.

8. **Clase Higlighter**

- **Función principal:** Aplica estilos visuales al código en el editor.
- **Responsabilidades:** Mediante el uso de expresiones regulares, define reglas de resaltado y estilos para palabras clave, nombres declarados, comentarios y cadenas.
- **Interacciones:** Interactúa con AreaCodigo para resaltar el código según las reglas.

9. **Clase NameChecker**

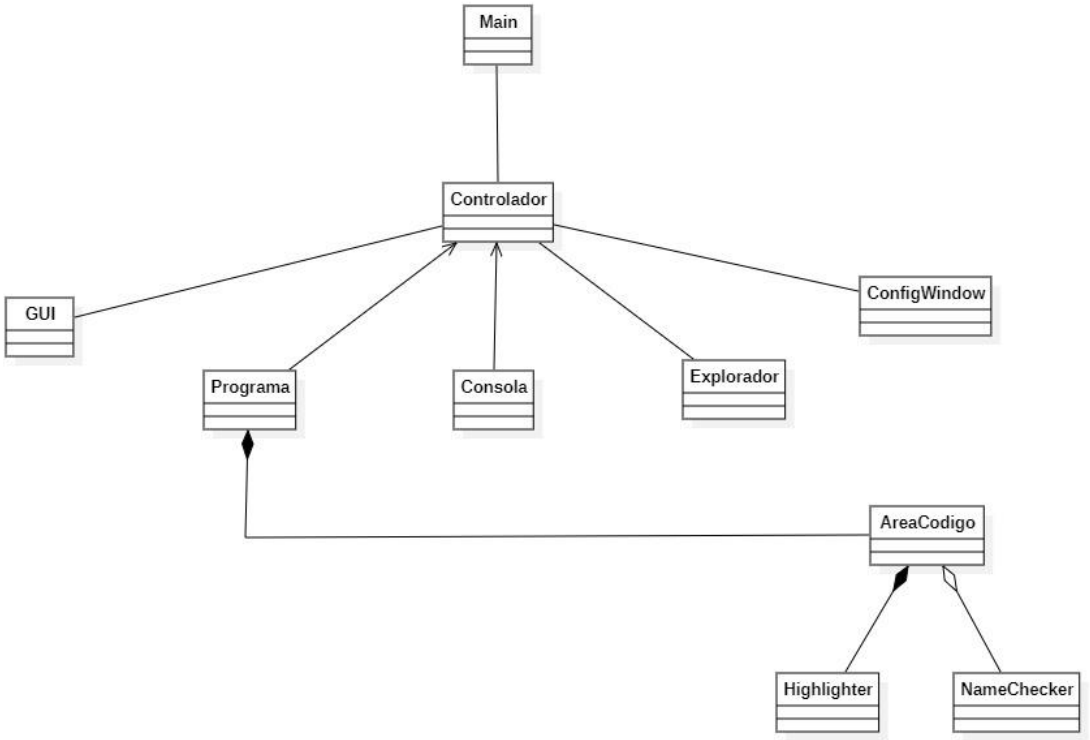
- **Función principal:** Visita los nombres en el árbol sintáctico del código.
- **Responsabilidades:** Maneja las apariciones de nombres en:
  - Declaraciones de variables, clases y funciones.
  - Sentencias.
- **Interacciones:**
  - AreaCodigo conecta el código con NameChecker
  - Controlador llama al módulo ast de Python y recupera el NameChecker desde el AreaCodigo para iniciar el visitado del árbol sintáctico del código.

10. **Clase ConfigWindow**

- **Función principal:** Ventana de configuración.
- **Responsabilidades:** Gestionar las configuraciones del entorno:
  - Tema (Claro/Oscuro).
  - Directorio raíz del explorador.

Diagrama de Arquitectura

Se ha realizado el diseño de un diagrama de clases en **UML**[xii] que representa las diferentes clases de las que se compone un programa y sus relaciones entre sí:



Flujo de Datos

A continuación, se detalla cómo los datos se mueven entre los componentes principales del sistema:

1. **Inicialización**
   - **Origen:** Main y Controlador
   - **Destino:** GUI, Explorador y ConfigWindow
   - **Datos:** Carga inicial de configuración de la interfaz:
     - Ventana de configuración, tema y directorio raíz predeterminados
     - Directorio raíz del explorador y montaje del sistema de archivos
     - Enables de las acciones de los menús
     - Conectar las acciones con los métodos

2. **Gestión de Archivos**
   - **Origen:** El usuario interactúa con Explorador
   - **Destino:** Controlador
   - **Datos:** Datos de los ficheros y carpetas del sistema de archivos relacionados con las operaciones de Explorador:
     - Índices del sistema de archivos para la navegación
     - Ruta del archivo a abrir, eliminar (mover a la papelera) o renombrar desde el explorador

3. **Gestión de la Configuración**
   - **Origen:** (ConfigWindow) El usuario interactúa con la ventana de configuración
   - **Destino:** Controlador
   - **Datos:** Se aplican, si los hubiera, los cambios seleccionados en la ventana

4. **Abrir un Archivo Python**
   - **Origen:** (Controlador) El usuario crea un nuevo programa o abre uno existente, ya sea interactuando con el menú de la ventana principal, atajo de teclado o seleccionandolo desde el explorador
   - **Destino:** Programa y AreaCodigo
   - **Datos:** Se conecta la señal de cambio en el texto y:
     - Si Nuevo: Atributos para AreaCodigo (tema a aplicar: claro u oscuro, y nombre del widget). Contenido y ruta vacíos.
     - Si Existente: Atributos para AreaCodigo más la ruta. Carga el código en AreaCodigo.

5. **Guardar o Guardar como un Archivo Python**
   - **Origen:** Controlador
   - **Destino:** Sistema de archivos
   - **Datos:** Se guarda el programa con su código fuente en el directorio raíz o deseado.

6. **Cerrar un Archivo Python**
   - **Origen:** Controlador
   - **Destino:** GUI
   - **Datos:** Pestaña del programa eliminada de la GUI

7. **Edición de Código**
   - **Origen:** El usuario edita código en el widget AreaCodigo
   - **Destino:** Programa, Controlador y Consola
   - **Datos:**
     - Programa: Atributo modificado
     - Enables del menú editar
     - Se dispara tras un timer el análisis estático del código: Si se detecta un error de sintaxis, a parte de resaltarlo en el código, se crea una consola que muestra el número de línea donde se ha producido el error y el mensaje de error.

8. **Ejecución**
   - **Origen:** (Controlador) El usuario pulsa el botón Ejecutar
   - **Destino:** Sistema de archivos y Consola
   - **Datos:**
     - El programa se guarda primero en el sistema de archivos si tiene cambios reflejados
     - Se inicia el proceso de ejecución del programa
     - Salida del programa en la ventana de resultados (Consola)
     - Error, si no hay un programa abierto o mensaje de error que ha lanzado el programa en la ventana de resultados

9. **Depuración**
   - **Origen:** (Controlador) El usuario pulsa el botón Debug
   - **Destino:** Sistema de archivos y Consola
   - **Datos:**
     - El programa se guarda primero en el sistema de archivos si tiene cambios reflejados
     - Se inicia el proceso de depuración del programa
     - Se crea una Consola a la que se añaden unos botones para las operaciones de depuración habituales (resumir ejecución, siguiente línea, step, stop)
     - El usuario navega por la ejecución paso a paso con estos botones
     - Salida del programa en la ventana de resultados (Consola)
     - Error, si no hay un programa abierto o mensaje de error que ha lanzado el programa en la ventana de resultados

Codificación

Este capítulo aborda la implementación de los componentes del sistema junto con las estrategias de pruebas para garantizar su correcto funcionamiento.

GUI

Es una clase generada con QtDesigner, contiene la definición de la ventana principal de la interfaz gráfica del IDE.
Este componente no requiere de lógica, se limita a la presentación. Diseño y distribución de widgets, menús y áreas de trabajo.

Métodos

- **setupIU():** Definición de la ventana principal y de los widgets.
- **retranslateUi():** Renombrado del texto de los distintos botones y acciones y enlace con sus atajos de teclado.

Controlador

Clase controlador de la ventana principal del IDE.

Atributos

- **programas:** Estructura de datos lista que mantiene los programas abiertos.
- **consolas**: Estructura de datos lista que mantiene las consolas abiertas.
- **contadorWidget:** Contador para constructores de widgets (nombres).
- **contadorDocumento:** Contador para el número de “nuevos programas” abiertos, ej: Sin-Titulo-1, Sin-Titulo-2, etc.
- **process:** (PyQt: QProcess) Para ejecutar los programas y comunicarnos con ellos.
- **filtros:** La extensión de los archivos que queremos que aparezcan en nuestro explorador lateral ('*.py').
- **pathRaiz:** El path raíz del explorador.
- **indexRaiz:** (QModelIndex) El índice raíz del modelo del sistema de archivos del explorador.
- **pathDirectorios:** Estructura de datos lista que mantiene los path de los directorios en los que vamos navegando para volver al superior con el botón "Back".
- **claridad:** Tema (claro/oscuro) por defecto para la inicialización del IDE.
- **timer:** (PyQt: QTimer) Temporizador para disparar el análisis estático del código.

Métodos

- **__init__():** Inicializa los atributos y la interfaz de usuario, monta el modelo de archivos (QFileSystemModel) y llama a setupConfiguración(), setupExplorador(), setupEnables() y setupConnects().
- **setupConfiguracion():** Se crea la ventana de configuración, se conectan sus señales y se añade al menú de la ventana principal.
- **abrir_configuracion():** Abre la ventana de configuración.
- **cambiar_configuración():** Disparado para reflejar los cambios realizados en la ventana de configuración. Llama a switch_estilo() y a switch_path().
- **switch_estilo():** Cambia el estilo a todas las AreaCodigo abiertas.
- **switch_path():** Cambia el Path Raíz del IDE y cambia el explorador para reflejar los cambios.
- **setupExplorador():** Crea el explorador e introduce 2 botones, Raíz y Back, para navegación.
- **explorador_clickado():** Llama a abrir_programa() con el path del fichero seleccionado al hacer doble click.
- **explorador_indiceRaiz():** Vuelve al directorio raíz en el explorador, disparado al pulsar el botón “Raíz”.
- **explorador_indiceBack():** Vuelve al directorio superior en el explorador, disparado al pulsar el botón “Back”.
- **explorador_getPath().**
- **explorador_getRelPath().**
- **setupEnables():** Enables predeterminados de las acciones de los menús al iniciar la aplicación.
- **tab_movida():** Disparado al mover una pestaña, cambios en el índice de programas para que coincida con el índice del tabWidget.
- **tab_cambiada():** Disparado al cambiar de pestaña de programa, recupera el índice de la pestaña a la que hemos cambiado y llama a enable_editar().
- **acciones_setEnabled():** Cambiar el enable de distintas acciones (Guardar, Pegar, Seleccionar todo, etc.), disparado en función de si tenemos programas abiertos o no.
- **enable_editar():** Actualiza los enables disponibles del menú editar en función del programa que tenemos abierto.
- **setupConnects():** Conecta los eventos y acciones de la ventana principal con los métodos correspondientes.
- **nuevo_programa():** Disparado al seleccionar la acción Nuevo, crea el objeto programa y lo añade a programas, añade la pestaña al tabWidget y llama a conectarArea_codigo().

- **abrir_programa():** Disparado al seleccionar la acción Abrir, Ejecuta las mismas acciones que al crear uno Nuevo y además carga en su area_codigo los contenidos del archivo fuente y llama a set_contenidoOriginal(). Si el programa ya estaba abierto se cambia el foco a éste.
- **conectarArea_codigo():** Se conecta la acción cuando cambia el texto (código) para llamar a programa_modificado().
- **programa_modificado():** Cambios en el atributo modificado de Programa en función de si es Nuevo o Existente, en cuyo caso cambia el nombre de la pestaña para reflejar cambios no guardados. Llama a enable_editar() y dispara el timer para llamar a parsear_codigo().
- **parsear_codigo():** Limpia errores previos y parsea en busca de errores de sintaxis, si no hay errores de sintaxis se visitan los nodos (nombres)y se buscan nombres que se utilizan pero no han sido declarados y llama para resaltarlos (amarillo) al encontrar un error de sintaxis se detiene y llama para resaltar el error (rojo). Llama a limpiar_errores(), resaltar_nombreNoDeclarado() y resaltar_error().
- **ejecutar_programa():** Disparado al pulsar el botón “Ejecutar”, si no hay programas abiertos muestra un mensaje de error por la ventana de resultados (consola), en caso contrario muestra la salida o error estándar por la ventana de resultados. Llama crear_consola(), mostrar_salida() y mostrar_error() para mostrar estos resultados y a cerrar_consola() para que haya solo una instancia de ejecución por programa.
- **depurar_programa():** Disparado al pulsar el botón “Debug”, si no hay programas abiertos muestra un mensaje de error por la ventana de resultados (consola), en caso contrario muestra la consola de depuración con los botones (resumir ejecución, next, step y stop) que se conectan a los métodos de la ejecución paso a paso con pdb (debugger de Python).
- **Métodos de los botones de depuración:** acciones de pdb y salida a la ventana de resultados.
- **cerrar_consola():** Al cerrar la pestaña se elimina el objeto de consolas.
- **cerrar_programa():** Al cerrar la pestaña se elimina el objeto de programas. Llama a mostrar_diálogo() para confirmar que se quiere cerrar si existen cambios sin guardar.
- **guardar_programa().**
- **guardarComo_programa().**
- **guardarTodos_programas().**
- **Métodos de las acciones del menú editar.**
- **Métodos para el evento de cierre:** Para confirmar que se desea cerrar la aplicación si existen cambios sin guardar para cada programa.

Programa

Contiene los atributos y métodos para manejar la información de los programas abiertos.

Atributos

- **tab:** Cada programa está asociado a una pestaña del tabwidget de la ventana principal.
- **verticalLayout:** Layout que contiene a AreaCodigo.
- **area_codigo:** Objeto AreaCodigo.
- **contadorWidget:** Utilizado para nombres (p.e: tab_1, area_codigo2)
- **modificado:** Booleano para controlar si el programa ha sido modificado.
- **ruta_archivo:** ruta absoluta al archivo abierto, si es nuevo está vacía.
- **contenido_original:** Contenido del archivo abierto, si es nuevo está vacío.

Métodos

- **__init__():** set de los atributos, creación de AreaCodigo y añadido al layout y pestaña.
- **es_contenidoOriginal():** Compara el atributo con el código actual en AreaCodigo.
- **set_contenidoOriginal().**

AreaCodigo

Widget editor de texto extendido (PyQt: QPlainTextEdit) contiene la clase del widget NumeroDeLinea.

Atributos

- **nameChecker:** visitador de nombres NameChecker
- **resaltado_sintaxis:** resaltador de tokens (expresiones regulares) Highlighter.

Métodos

- **__init__():** Creamos el Highlighter y llamamos a highlight_code() para que Highlighter apunte a nuestro documento, creamos el widget NumeroDeLinea, conectamos sus señales y inicializamos la anchura del widget.
- **setEstilo():** Aplica estilo claro u oscuro a AreaCodigo
- **highlight_code():** El contenido de AreaCodigo es apuntado por Highlighter.
- **construir_nameChecker():** Crea el objeto NameChecker y lo devuelve.

- Eventos de menú contextual, atajos de teclado y del widget NumeroDeLinea y Slots de señales del widget NumeroDeLinea.

Consola

Contiene atributos para manejar la información de los programas en ejecución y el widget de la ventana de resultados.

Atributos

- **tab:** Cada ventana de resultados está asociada a una pestaña del tabwidget de la zona inferior de la ventana principal.
- **verticalLayout:** Layout que contiene la consola.
- **consola:** (PyQt: QTextEdit) Ventana de resultados.
- **contadorWidget:** Utilizado para nombres (p.e: tab_1, area_codigo2)
- **ruta_archivo:** ruta absoluta del programa en ejecución.

Métodos

- **__init__():** Creación de la consola.

Explorador

Widget explorador extendido (PyQt: QTreeView).

Atributos (PyQt: QAction)

- **modelo:** (PyQt: QFileSystemModel) Modelo del sistema de archivos.
- **accionAbrir:** Acción para abrir ficheros desde el menú contextual del explorador
- **accionPath:** Acción para copiar el path absoluto del fichero desde el menú contextual del explorador
- **accionPathRel:** Acción para copiar el path relativo del fichero desde el menú contextual del explorador
- **accionEliminar:** Acción para Eliminar el fichero desde el menú contextual del explorador
- **accionRenombrar:** Acción para renombrar el fichero desde el menú contextual del explorador

Métodos

- **__init__():** Definimos el modelo del sistema de archivos del explorador.
- **mover_a_papelera():** Utiliza la función send2trash. Muestra un mensaje de confirmación antes de eliminar el fichero o directorio seleccionado.
- **renombrar():** Utiliza la librería os. Muestra un diálogo para ingresar el nuevo nombre y se renombra.
- **contextMenuEvent():** Menú para el evento del menú contextual.
- **getModelo().**

Highlighter

Resaltador de sintaxis extendido (PyQt: QSyntaxHighlighter).
Existe un Highlighter2 pero se incluye solo para destacar ciertas cosas de la ventana de resultados, especialmente en modo depuración.

Atributos

- **builtins:** Funciones y constantes predefinidas como print, len o True/False, etc.
- **keyWords:** Palabras reservadas del lenguaje como import, las sentencias, class y def.
- **braces:** Paréntesis, Llaves y Corchetes.

Métodos

- **__init__():** Se define un formato para las keyWords, builtins, braces, literales como class, self y def, los nombres en la declaración de funciones, strings, números y comentarios.
- **_mappings:** Estructura de datos que contiene las expresiones regulares que queremos capturar, el índice desde donde empieza y el formato que queremos aplicar como una tupla en cada posición, se reconstruyen como QRegularExpresion para que las reconozca highlightBlock().
- **highlightBlock():** Resalta el texto (código) del AreaCodigo de un programa, es llamado cada vez que se produce un match (expresiones regulares) y se necesita resaltar texto.
- **match_multiline():** Maneja el resaltado de los comentarios multilínea.

NameChecker

Visitador de nodos extendido (NodeVisitor) del árbol sintáctico abstracto (ast).

Atributos

- **variables:** lista de nombres de variables, funciones, etc. declaradas.
- **nombres_importados:** lista de nombres de importaciones (import).
- **clases_definidas:** lista de nombres de clases declaradas.
- **errors:** lista de errores, cuando se usa un nombre pero éste no ha sido declarado previamente.

Métodos

- **__init__():** Inicializa los nombres de variables con los predefinidos y le añade 'self' e inicializa a vacio los nombres importados, las clases definidas y los errores.
- **visit_Assign():** Maneja las declaraciones de variables normales
- **visit_AnnAssign():** Maneja las declaraciones de variables con asignación.
- **visit_For():** Maneja nombres declarados dentro de sentencias ‘for’.
- **visit_ClassDef():** Maneja declaraciones de nombre de clase definida.
- **visit_Call():** Verifica si una llamada a constructor es de una clase definida.
- **visit_FunctionDef():** Maneja nombres de variables locales, parámetros y argumentos en definiciones y llamadas de métodos.
- **visit_With():** Maneja declaraciones en sentencias 'with'.
- **visit_Import():** Maneja nombres de importaciones tipo: 'import ast'.
- **visit_ImportFrom():** Maneja nombres de importaciones tipo: 'from os import path'.
- **visit_Name():** Maneja el resto de nombres, si se está utilizando y si no está en las listas de declaraciones variables o importaciones, se añade a la lista de errores que luego se usará para subrayar de amarillo los nombres no declarados.

ConfigWindow

Ventana de diálogo extendida (PyQt: QDialog) consta de:

- **Botones** Aceptar, Cancelar y "..." (Para abrir una ventana y seleccionar nuevo path raíz del explorador).
- **Etiquetas** para mostrar el contexto actual.
- **ComboBox** para elegir el tema (claro/oscuro).

Atributos

- **path_Label:** (PyQt: QLabel) Etiqueta para cambiar el contenido que muestra el pathRaiz actual.
- **pathRaiz:** Path raíz actual del explorador.
- **boxActual:** Contenido actual seleccionado del comboBox
- **cambiosReflejados:** (PyQt: pyqtSignal) Señal para el Controlador que manda para actualizar los cambios (Índice Raíz del explorador y tema).

Métodos

- **__init__():** Inicializa los atributos y llama a setupIU().
- **setupIU():** Definición de la ventana de configuración y de los widgets. También conecta las acciones al pulsar los botones y cuando cambia el contenido del comboBox.
- **open_file_dialog():** Abre una ventana de diálogo al pulsar el botón "..." para seleccionar un nuevo directorio raíz para el explorador.
- **accept():** Emite la señal cambiosReflejados a Controlador con los datos a actualizar y cierra la ventana.
- **reject():** Cierra la ventana sin aplicar cambios.
- **set_boxActual().**

Testing y Pruebas

En este proyecto se han realizado **pruebas unitarias**[xiii] ya que gracias a ellas se puede mejorar la calidad del proyecto ya que garantizan que el código fuente funciona de manera correcta, además mejoran los tiempos de desarrollo ya que garantiza que modificar el código fuente no hace que el programa funcione incorrectamente.

Para ello se ha hecho uso de la librería **unittest** que permite crear varios tests que verifiquen que una función actúa correctamente.
Estos tests se han realizado mediante la creación de clases de Test que heredan de unittest.TestCase y dentro de las cuales se definen diversos tests como métodos de clase. En cada uno de estos tests se ejecutan las comprobaciones necesarias utilizando assertEqual.

El resultado de estos tests es OK si la prueba se ha ejecutado correctamente, FAIL, si la prueba no se ha ejecutado correctamente pero el error se produce de forma controlada y ERROR si la prueba no se ejecuta correctamente y el error es una excepción no controlada en la prueba. Gracias a estos resultados se ha podido realizar una depuración correcta del proyecto.

También se han realizado **pruebas de integración**[xiv] después de haberse aprobado las pruebas unitarias, que han permitido probar la interoperabilidad entre los componentes, es decir, que todos los elementos unitarios de los que está compuesto este proyecto software funcionan juntos correctamente.

Para ello se ha vuelto a utilizar unittest después de realizar un análisis en profundidad del flujo del código del proyecto, para así poder seleccionar los escenarios más relevantes, configurar un entorno de prueba que permita realizar pruebas representativas y confiables y finalmente definir los casos de prueba.
Estas pruebas se han ido actualizando con cada nuevo prototipo hasta obtener finalmente los casos de prueba de todos los componentes del sistema.

Una vez en este punto se han realizado también pruebas manuales del sistema simulando ser un usuario del programa que no conoce el sistema para así comprobar su completa adecuación a los requisitos definidos para este tipo de soluciones software.

Conclusiones

El desarrollo “IDE específico para Python que facilite las operaciones fundamentales de la edición de software” ha concluido con éxito, cumpliendo los objetivos planteados y ofreciendo una solución funcional y accesible que cumple con las funcionalidades esenciales.

Aún con la complejidad que planteaba este proyecto, decidí que quería abarcar un lenguaje de programación desconocido del que obtener experiencia. Me decanté por Python debido a su fama y versatilidad y he encontrado el desarrollo de este proyecto como una experiencia muy enriquecedora que me hace considerar Python como un lenguaje muy cómodo y con muchas utilidades como por ejemplo, las funciones integradas en los tipos como String.
También destacar que no había trabajado a este nivel con interfaces gráficas, de lo cual también me llevo una experiencia muy útil para mi carrera profesional.

Cumplimiento de los objetivos clave:

1. **Interfaz gráfica. Cumplido:** es simple e intuitiva, con una disposición clara de los elementos, lo que reduce significativamente la curva de aprendizaje, incluso para usuarios novatos.

2. **Editor de Código. Cumplido:** Proporciona soporte para resaltado de sintaxis y de errores en tiempo real que mejoran la escritura de código.

3. **Funciones de Archivo y Edición habituales. Cumplido:** El entorno cuenta con las opciones: Nuevo, Abrir, Guardar, Guardar como, Guardar todo, Cerrar, Cerrar todo, Deshacer, Rehacer, Cortar, Copiar, Pegar y Seleccionar todo.

4. **Manejo de ventanas múltiples (edición y resultados). Cumplido:** Posibilidad de cambiar entre pestañas de edición y ejecución de programas distintos.

5. **Depurador Paso a Paso. Cumplido:** Ejecuta el código paso a paso, indica la línea que se está ejecutando en este momento por consola, redirige los resultados o errores a la ventana de resultados y cuenta con las funciones de depuración habituales, resumir ejecución, siguiente línea, step y stop.

6. **Gestión de Proyectos. Cumplido:** El explorador facilita la navegación por proyectos de Python, añadiendo la posibilidad de volver a la carpeta superior o al directorio raíz y ofrece operaciones básicas como abrir los programas, renombrarlos, eliminarlos y copiar su ruta.

7. **Ejecución del Código y Ventana de Resultados. Cumplido:**
   Cuenta con la salida de ejecución y mensajes de error, añadiendo errores que se pueden dar usando el entorno, en un espacio diferenciado.

Seguimiento de la Planificación

- **Semana 1:**
  Se desarrollaron los prototipos 1 y 2 sin complicaciones, el uso de **Qt Designer** facilitó en gran medida la elaboración del prototipo de interfaz gráfica. Para el Editor de Código, solo se tomaron las decisiones de diseño sobre la abstracción en la clase Programas.

- **Semana 2:**
  Surgieron una serie de contratiempos, referidos sobre todo al comportamiento de los enables de los menús al interactuar con la interfaz.

- **Semana 3 y 4:**
  Para la ejecución de programas surgieron complicaciones para la decisión del planteamiento, se probaron distintas formas de implementación hasta que se encontró la manera gracias al uso de QProcess.

- **Semana 4 y 5:**
  El módulo ast de Python facilitó mucho esta tarea, se exploraron varios ejemplos proporcionados en distintos **sitios web**[xv] para dar con las distintas formas de capturar todos los nombres deseados para el resaltado de los nombres no declarados.
  Se decidió incluir NameChecker en AreaCodigo para crearlo en el momento del análisis estático. La implementación se alargó debido a las pruebas necesarias para alcanzar el comportamiento deseado.

- **Semana 5 y 6:**
  Se encontraron ejemplos muy útiles que facilitaron la decisión y la implementación del widget NúmeroDeLínea.

  Para el resaltado de sintaxis se estudiaron expresiones regulares, la implementación se alargó un poco por esta parte, debido a las pruebas de ensayo y error para conseguir el correcto resaltado de las expresiones que se querían capturar.

  Para el explorador no hubo mayor complicación, gracias sobre todo a la experiencia obtenida durante el desarrollo.

- **Semana 6 y 7:**
  Se implementaron los distintos menús contextuales para los widgets deseados sin dificultad, al implementar el del explorador se terminaron de implementar también el resto de sus operaciones.

  La implementación de la ventana de configuración se complicó un poco debido a que era la primera vez que se experimentaba con la creación de una señal personalizada.

- **Semana 8 y 9:**
  Surgieron diversas complicaciones con la decisión del planteamiento y la implementación, ya que se volvía a empezar después intentar con los distintos enfoques. Finalmente se desechó el intento de integrar un compilador integrado en el IDE y se alcanzó finalmente el comportamiento deseado usando **PDB** (el debugger de Python) y controlando su entrada/salida.

- **Semanas 10 y 11:**
  El progreso de la memoria tuvo un avance más significativo durante la segunda semana.

Eficiencia: Se mantiene un rendimiento fluido, incluso en sistemas con recursos limitados, debido a la arquitectura ligera y modular. La separación en una misma vista de las áreas de edición, consola de resultados y explorador de proyectos mejoran la productividad.

Comparativa con soluciones existentes

1. PyCharm

- **Ventajas de PyCharm frente a este IDE:**
  - Soporte avanzado para aplicaciones profesionales y proyectos complejos.
  - Integración nativa con frameworks, herramientas de control de versiones y pruebas automatizadas.
  - Extensas características para desarrollo web y ciencia de datos.

- **Desventajas de PyCharm frente a este IDE:**
  - Alto consumo de recursos, lo que lo hace menos ideal para usuarios con hardware limitado.
  - Curva de aprendizaje elevada para principiantes.
  - Modelo de licenciamiento que puede ser una barrera para usuarios sin presupuesto.

- **Conclusión Comparativa:**

  Este IDE se enfoca en un público más amplio, incluidas personas con recursos limitados o sin experiencia avanzada. Ofrece funcionalidades esenciales sin la complejidad o el costo de PyCharm.

2. Spyder

- **Ventajas de Spyder frente a este IDE:**
  - Excelente para proyectos de ciencia de datos, con integración de visualización y exploración interactiva de variables.
  - Ligero y bien adaptado para tareas de análisis numérico.

- **Conclusión Comparativa:**

  Si bien Spyder es fuerte en tareas científicas, nuestro IDE aborda de manera más general el desarrollo de software en Python, ofreciendo herramientas para edición, depuración y ejecución, siendo más adecuado para desarrolladores generales y principiantes.

Valoración Final

El IDE desarrollado logra un equilibrio entre funcionalidades esenciales y simplicidad. Es una solución que:
Responde a las necesidades de un público diverso, desde estudiantes hasta desarrolladores principiantes.
Mitiga las desventajas de soluciones robustas pero complejas como PyCharm y Spyder, centrándose en ser ligero y fácil de usar.

Mejoras y trabajos futuros

Como propuesta de mejora, una de las características más demandadas en este tipo de desarrollos es el autocompletado de código, una función muy útil sobre todo para usuarios con poca experiencia, que no se ha implementado por escapar a la complejidad y alcance de este proyecto. También se podría añadir la implementación de un compilador integrado para realizar las operaciones de depuración y análisis estático del código.

En el futuro podría ampliarse con características como integración de control de versiones y soporte básico para proyectos de ciencia de datos o desarrollo web, así como soporte para más lenguajes de programación, garantizando así una evolución acorde con las demandas del mercado.

Referencias

[i] **IEEE Software.** (s.f.). *Development Tools: Analyzing the Components of IDEs*. IEEE. Recuperado de https://ieeexplore.ieee.org/

[ii] Ghuriani, V., Gautam, A., & Chawla, V. (2023, septiembre). *Python Programming: A Comprehensive Approach* (p. 22). PHI Learning Pvt. Ltd.

[iii] **JetBrains.** (s.f.). *PyCharm: El IDE de Python para la ciencia de datos y el desarrollo*. Recuperado de https://www.jetbrains.com/help/pycharm/python.html

[iv] **Spyder IDE.** (s.f.). *Spyder | The Python IDE that scientists and data analysts deserve*. Recuperado de https://spyder-ide.org y https://docs.spyder-ide.org/current/index.html

[v] **ISO.** (s.f.). *ISO 9241. Usabilidad*. Recuperado de https://www.icalia.es/w/usabilidad

[vi] ISO. (s.f.). *ISO/IEC 25010:2011 – Modelos de calidad del producto y de calidad en uso*. Recuperado de https://iso25000.com/index.php/normas-iso-25000/iso-25010/27-flexibilidad y https://iso25000.com/index.php/normas-iso-25000/iso-25010/26-mantenibilidad

[vii] Alonso Amo, F., Martínez Normand, L., & Segovia Pérez, F. J. (2005, febrero). *Introducción a la ingeniería del software* (p. 110). Delta Publicaciones.

[viii] Fitzpatrick, M. (2022). *Create GUI Applications with Python & Qt6* (p. 30). Fitzpatrick Martin.

[ix] **Qt Project.** (s.f.). *Documentación Qt for Python*. Recuperado de https://marketplace.visualstudio.com/items?itemName=seanwu.vscode-qt-for-python

[x] Chapman, S. N. (2006). *Planificación y control de la producción* (p. 181). Pearson Education.

[xi] Jiménez de Parga, C. (2021, abril). *UML. Arquitectura de aplicaciones en Java, C++ y Python* (2ª ed., p. 140). RA-MA S.A. Editorial y Publicaciones.

[xii] Debrauwer, L., & Van der Heyde, F. (2016). *UML 2.5: Iniciación, ejemplos y ejercicios corregidos*. Ediciones ENI.

[xiii] Moreno Muñoz, A., & Córcoles Córcoles, S. (2019, noviembre). *Python Práctico* (p. 76). RA-MA S.A. Editorial y Publicaciones.

[xiv] Guagliano, C. (2019, octubre). *Programación en Python II: Programación orientada a objetos* (p. 73). RedUsers.

[xv] Usuario: Houy Narun. (2020, 20 de enero). How to get actual user-defined variable name in Python abstract syntax tree (AST)?. Stack Overflow. Recuperado de https://stackoverflow.com/questions/59821113/how-to-get-actual-user-defined-variable-name-in-python-abstract-syntax-tree-ast